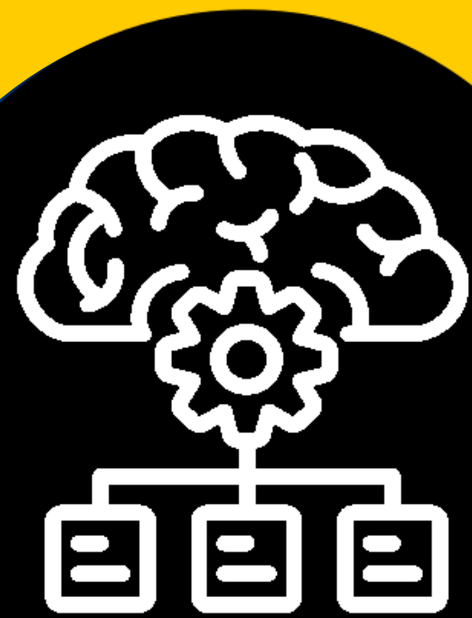
Deep Learning

Seq 2 Seq

Research Depth and Industry Skills



By:

Dr. Zohair Ahmed

www.youtube.com/@ZohairAI Subscribe

www.begindiscovery.com

What Is Sequential Data?

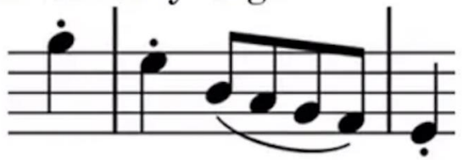
- **Sequential data** is any type of data where the **order of the elements matters**.
- The meaning of the data depends on the sequence in which it appears.
- Examples of sequences in daily life:
  - Words in a sentence
  - Notes in a song
  - Temperatures recorded every hour
  - Frames in a video
  - ECG or heart-rate signals
- If you shuffle the order, the meaning changes or becomes meaningless.
- **Sequence = order is important**

Examples of sequence data

- **Time Series Data**
  - This is a common type of sequential data where measurements are taken at regular intervals over time. Examples include stock prices, temperature readings, and sensor data.
- **Natural Language Text**
  - Textual data, such as sentences or paragraphs, is inherently sequential. The order of words matters to convey meaning, context, and relationships between words.
- **Speech Signals**
  - Audio signals from spoken language are sequential. Phonemes, syllables, and words are organized sequentially to form sentences.
- **Music:**
  - Musical notes in a composition are ordered sequentially to create melodies, harmonies, and rhythms.
- **DNA Sequences:**
  - In genetics, DNA sequences represent the arrangement of nucleotides in a specific order, carrying genetic information.
- **Video Frames:**
  - In video data, each frame is a snapshot of a scene at a specific time. The sequence of frames creates the illusion of motion.
- **User Actions / Clickstream data:**
  - User interactions on a website or application, such as clicks or keystrokes, occur sequentially and can be used to understand user behaviour.

Examples of sequence data

Examples of sequence data

Speech recognition		→	"The quick brown fox jumped over the lazy dog."
Music generation		→	
Sentiment classification	"There is nothing to like in this movie."	→	★☆☆☆☆
DNA sequence analysis	AGCCCCTGTGAGGAACTAG	→	AGCCCCTGTGAGGAACTAG
Machine translation	Voulez-vous chanter avec moi?	→	Do you want to sing with me?
Video activity recognition		→	Running
Name entity recognition	Yesterday, Harry Potter met Hermione Granger.	→	Yesterday, Harry Potter met Hermione Granger.

Andrew Ng

Recurrent Neural Networks (RNNs)

- Recurrent Neural Networks (RNNs) are a class of neural networks designed to process sequential data by retaining information from previous steps.
- They are especially effective for tasks where context and order matter.
  - Designed for sequential and temporal data
  - Maintains memory of past inputs
  - Widely used in NLP, forecasting and speech tasks

When Do We Need RNNs?

1. Past information affects the current output

- To predict the next word in a sentence, you must understand the previous words.

2. When data comes in a time-series form

- Predicting tomorrow’s temperature depends on previous days.

3. The model must "remember" previous inputs

- RNNs have a memory (hidden state) that stores information from earlier steps.

RNN

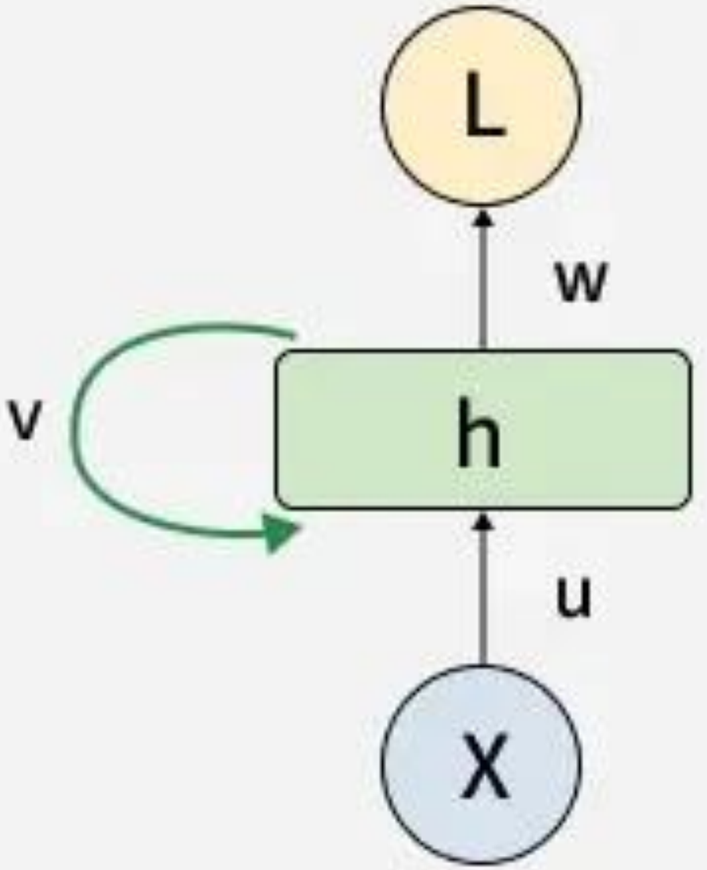
- Imagine reading a sentence and you try to predict the next word; you don’t rely only on the current word but also remember the words that came before.
- RNNs work similarly by **“remembering”** past information i.e. it considers all the earlier words to choose the most likely next word.
- This memory of previous steps helps the network understand context and make better predictions.

Key Components of RNNs

- There are mainly two components of RNNs that we will discuss.

1. Recurrent Neurons

- The fundamental processing unit in RNN is a Recurrent Unit**.**
- They hold a hidden state that maintains information about previous inputs in a sequence.
- Recurrent units can **"remember"** information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



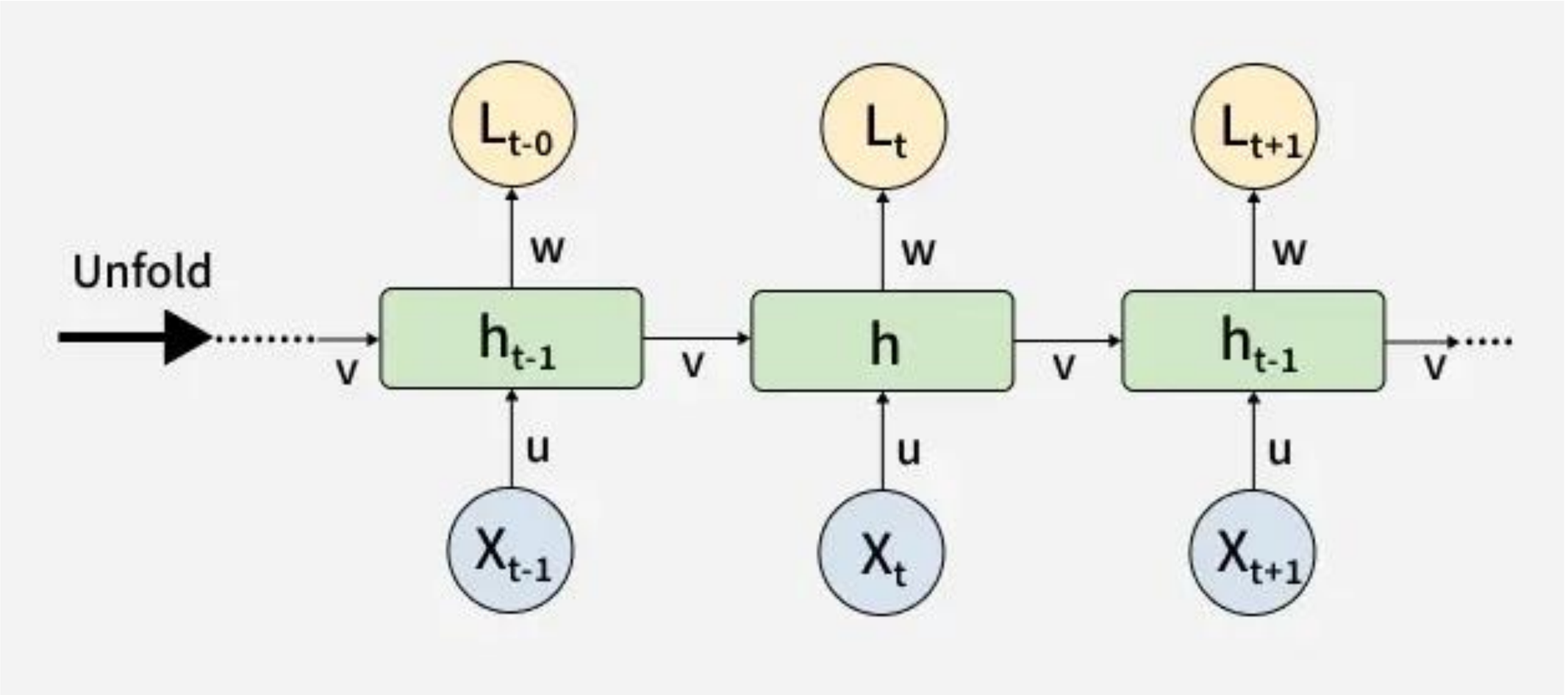
Key Components of RNNs

2. RNN Unfolding

- RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps.
- During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.
- This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.

Key Components of RNNs

2. RNN Unfolding



Example: Initial Input Weights

- **Choose Small Numbers for Weights**
- Let’s choose tiny numbers to keep calculations simple:
  - Wx = 0.5
  - Wh = 0.1
  - Wy = 1.0
  - bh = 0.0
  - by = 0.0
- Initial hidden state:
- h0 = 0

- Now let's unroll the RNN.
- **RNN Fully Unrolled Over 3 Time Steps**
- We will compute **h1, h2, h3** and **y1, y2, y3**.
- Where
  - h_t= hidden state at time t
  - x_t= input at time t
  - W_x, W_h, W_y= weights
  - b_h, b_y= biases
  - $\tanh$= activation function

Example: Step t = 1

- **Step t = 1**
- $h_1 = \tanh\left(W_x x_1 + W_h h_0\right)$
- **Substitute values:**
- $h_1 = \tanh\left(0.5 \cdot 1.0 + 0.1 \cdot 0\right)$
- $h_1 = \tanh\left(0.5\right)$
- **Compute:**
- $h_1 \approx 0.4621$
- **Output:**
- $y_1 = W_y \cdot h_1 = 1.0 \cdot 0.4621$
- $y_1 = 0.4621$
- **Memory updated**
- **Output generated**

Example: t=2

- **Step t = 2**
- $h_2 = \tanh(W_x x_2 + W_h h_1)$
- **Substitute:**
  - $h_2 = \tanh(0.5 \cdot 2.0 + 0.1 \cdot 0.4621)$
  - $h_2 = \tanh(1.0 + 0.04621)$
  - $h_2 = \tanh(1.04621)$
- **Compute:**
  - $h_2 \approx 0.7798$
- **Output:**
- $y_2 = 1.0 \cdot 0.7798 = 0.7798$

- RNN remembers **x1 + x2**

  Output produced

Example: Step t = 3

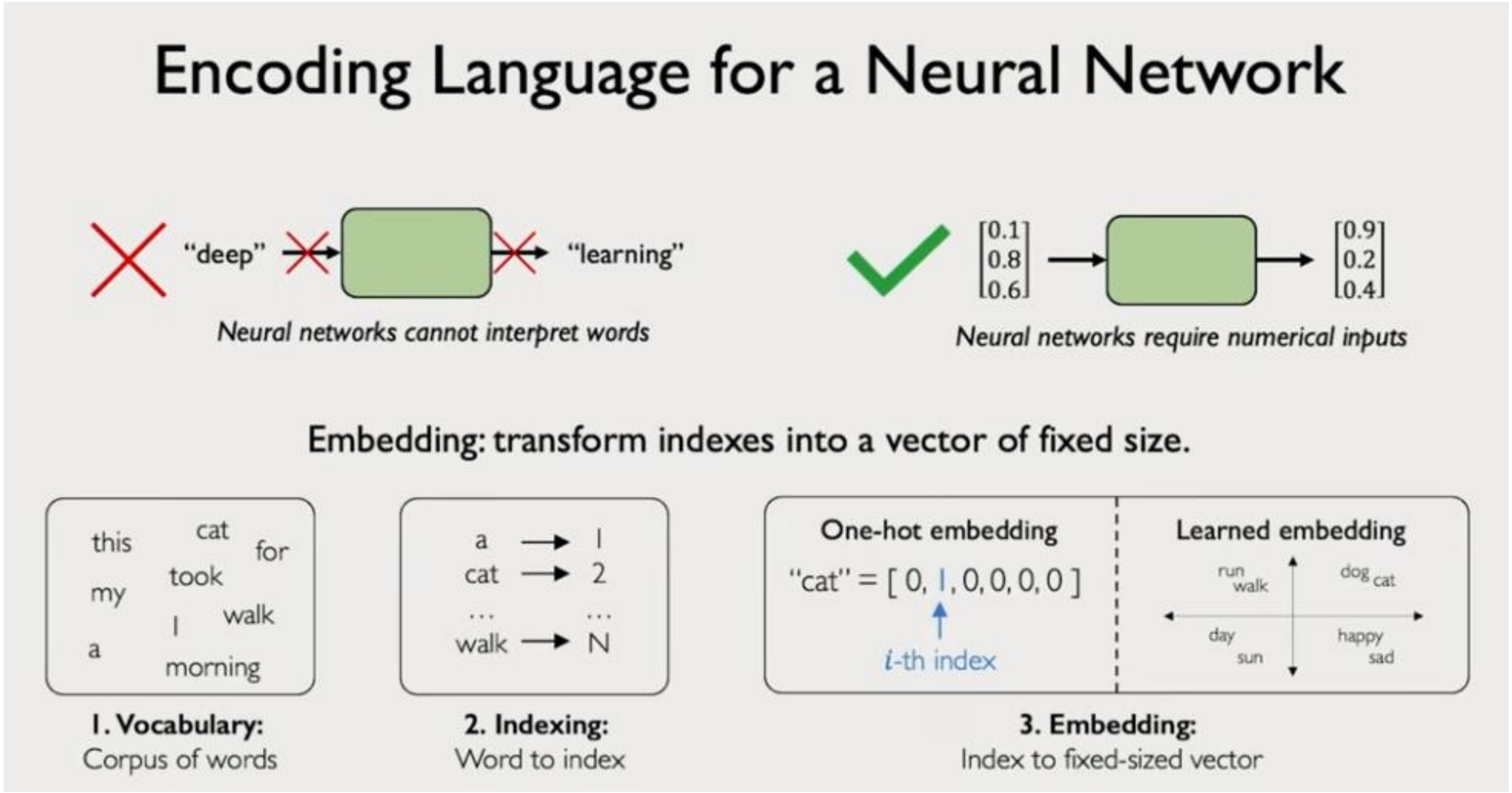
- **Step t = 3**
- **Formula:**
- $h_3 = \tanh\left(W_x x_3 + W_h h_2\right)$
- **Substitute:**
- $h_3 = \tanh\left(0.5 \cdot 3.0 + 0.1 \cdot 0.7798\right)$
- $h_3 = \tanh\left(1.5 + 0.07798\right)$
- $h_3 = \tanh\left(1.57798\right)$
- **Compute:**
- $h_3 \approx 0.9180$

- **Output:**
- $y_3 = 1.0 \cdot 0.9180 = 0.9180$
- RNN remembers **x1 + x2 + x3**
- Final output generated

Final Summary Table

Time Step	Input (xt)	Hidden State Formula	Hidden State (ht)	Output (yt)
t = 1	1.0	tanh(0.5×1 + 0.1×0)	0.4621	0.4621
t = 2	2.0	tanh(0.5×2 + 0.1×0.4621)	0.7798	0.7798
t = 3	3.0	tanh(0.5×3 + 0.1×0.7798)	0.9180	0.9180

Language Representation

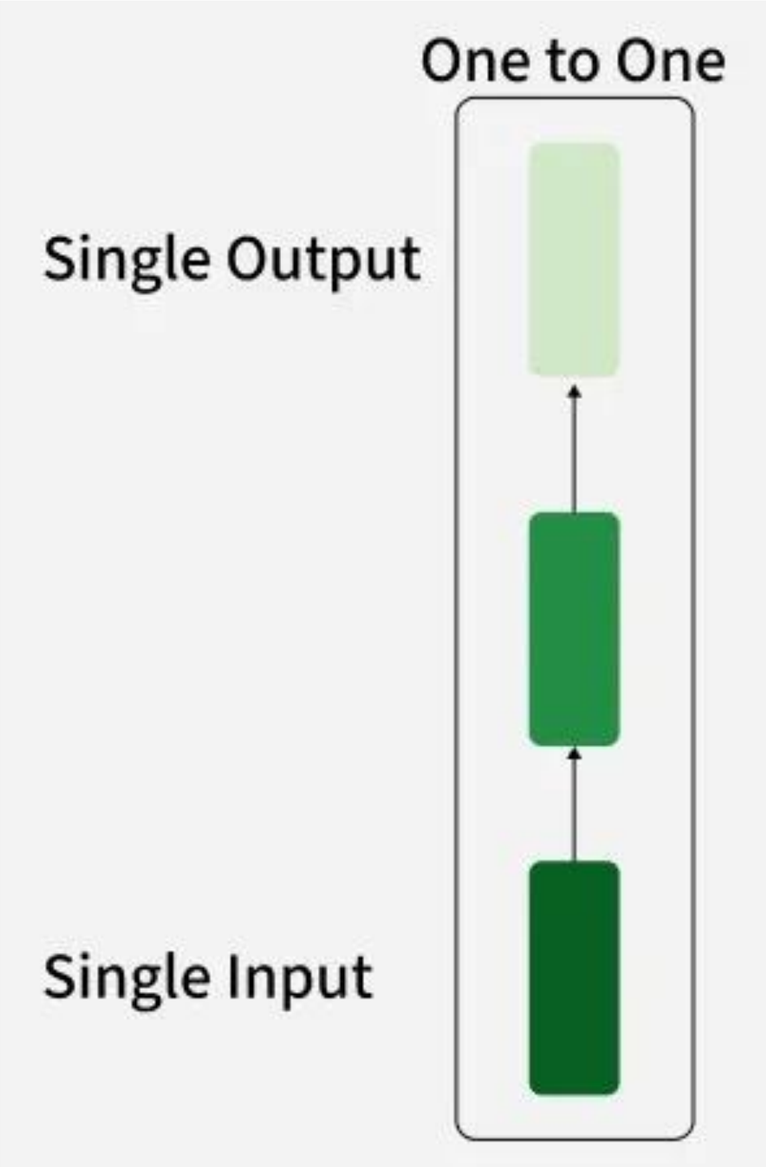


Types of Recurrent Neural Networks (RNNs)

- RNNs can be used in different ways depending on:
- How many inputs you give
- How many outputs you need
- Whether the inputs/outputs are sequences
- There are **4 types of RNN architectures**:
- One-to-One
- One-to-Many
- Many-to-One
- Many-to-Many

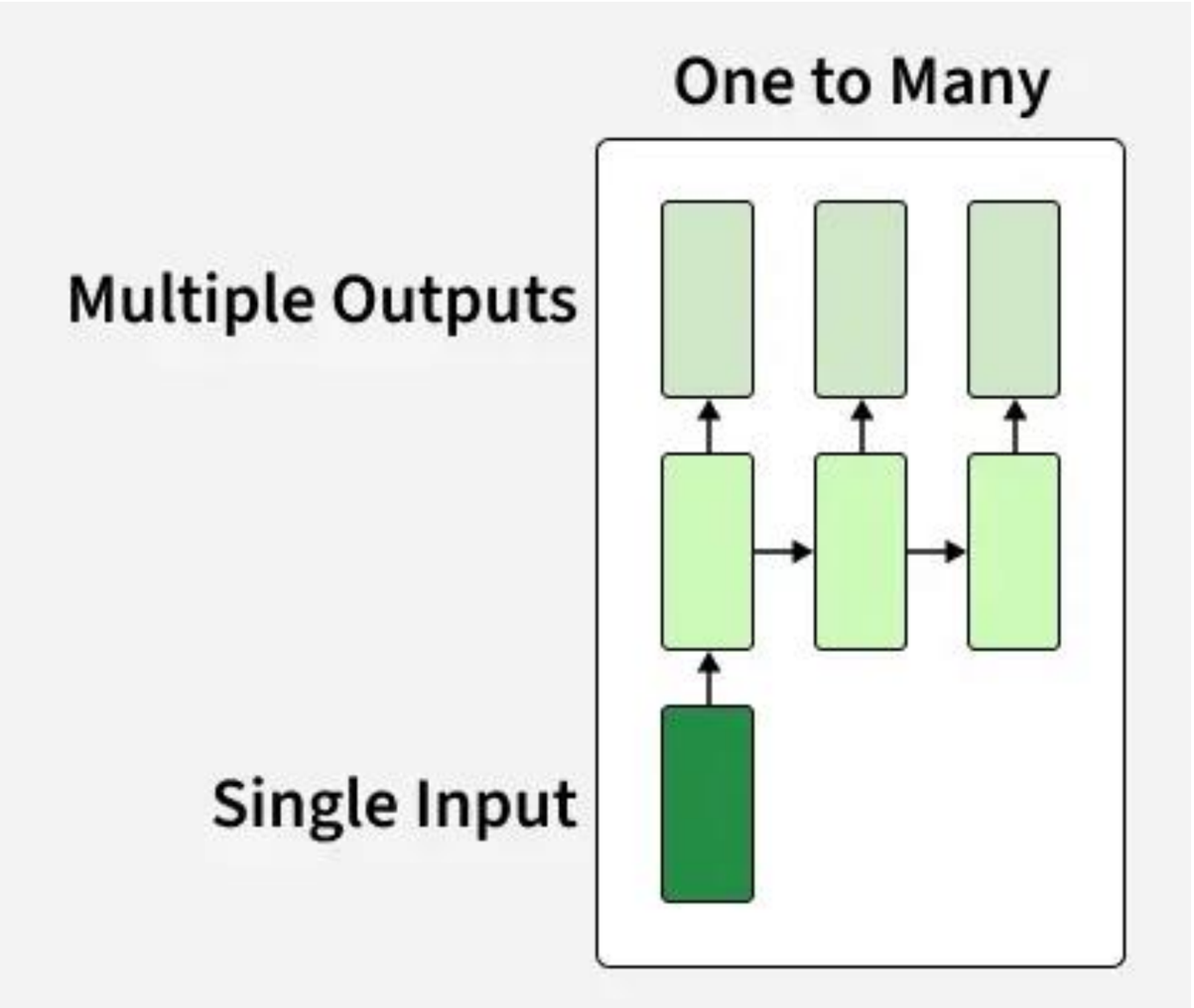
1. One-to-One RNN

- **ONE input**
- **ONE output**
- No sequence involved.
- **When used:**
- This is basically a **normal feedforward neural network** (not really a sequential RNN).
- **Example tasks:**
- Image classification (cat or dog)
- Basic binary classification
- Spam vs non-spam
- **Example:**
- Input: 1 image
- Output: 1 label (cat)
- **Why use:**
- Use when **order does not matter,** NOT sequential data.



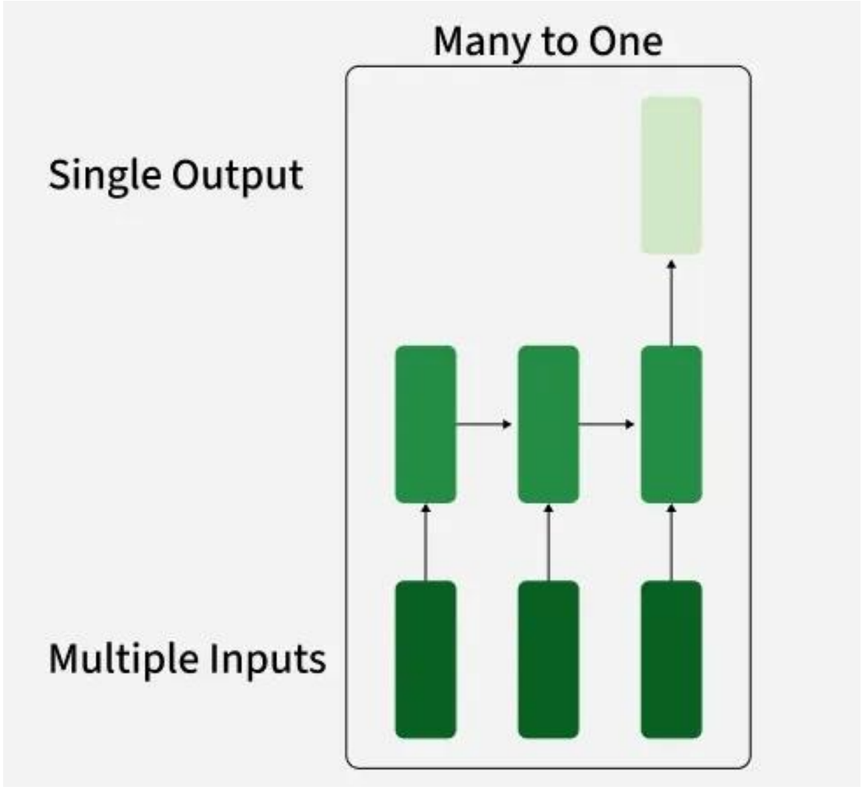
2. One-to-Many RNN

- **ONE input**
- **A sequence of outputs** generated over multiple time steps.
- **When used:**
- This is used when a **single input triggers a sequence**.
- **Example tasks:**
- **Image captioning**
  - (One image → multiple words in a caption)
- **Music generation**
  - (One start note → generate melody)
- **Story generation**
  - (One prompt → generate paragraph)
- **Example:**
- Input: 1 image
- Output: "A dog is running in the park."
- **Why use:**
- Use when **one object produces multiple time-based outputs**.



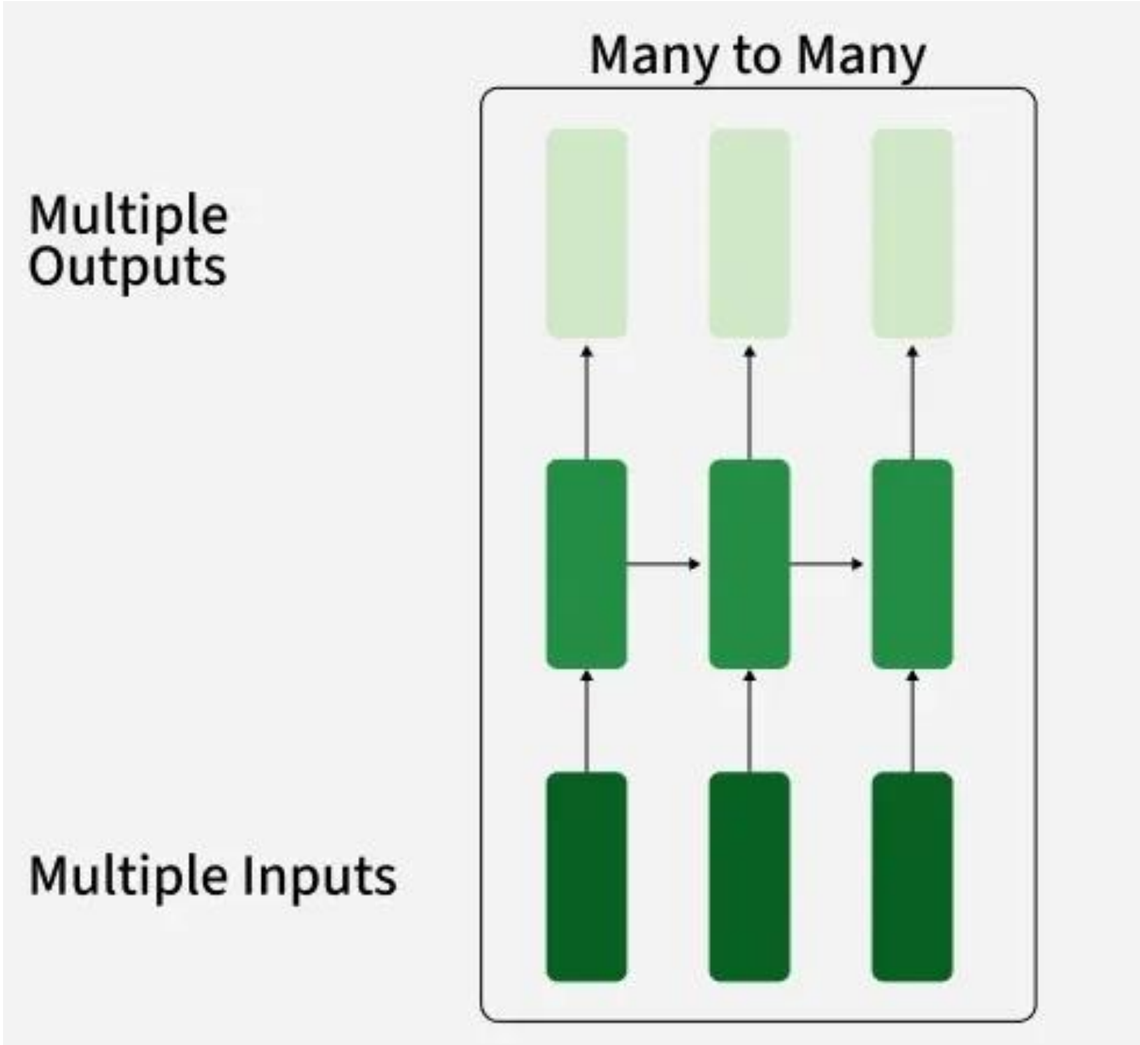
3. Many-to-One RNN

- **A sequence of inputs**
- **ONE output**
- Used when you need the **whole sequence** to make **one final decision**.
- **Example tasks:**
  - **Sentiment analysis** (Sentence → positive/negative)
  - **Spam detection** (Email text → spam/not spam)
  - **Sequence classification** (ECG signal → disease/healthy)
- **Example:**
- Input: ["I", "love", "this", "movie"]
- Output: Positive
- **Why use:**
- Use when **context across the whole sequence** matters for a single prediction.



4. Many-to-Many RNN

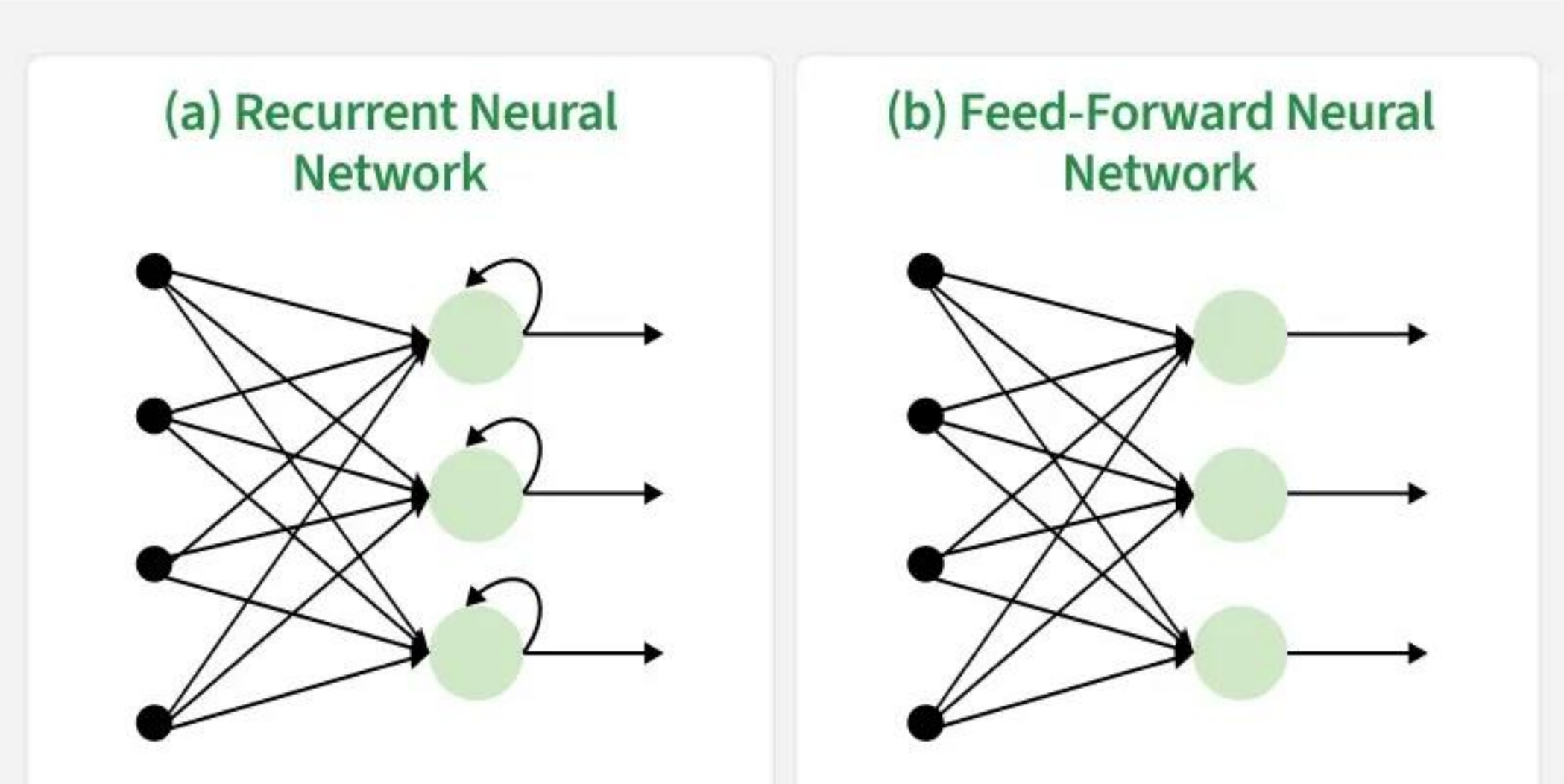
- **Sequence in**
- **Sequence out**
- **Two versions:**
- **Equal length input-output** (e.g., POS tagging)
- **Different length sequences** (e.g., machine translation)
- **Example tasks:**
  - **Machine translation** (English sentence → French sentence)
  - **Speech recognition** (Audio wave → text)
  - **Named Entity Recognition (NER)** (Word → label for each word)
- **Example:**
- Input: ["I", "love", "NLP"]
- Output: [PRON, VERB, NOUN]
- **Why use:**
- Use for **sequence transformation** tasks.



How RNN Differs from Feedforward Neural Networks?

- **Feedforward Neural Networks (FNNs)**
  - Process data in a single direction, from input to output
  - Do not store information from previous inputs
  - Suitable for tasks with independent data, such as image classification
  - Perform poorly on sequential data due to the absence of memory
- **Recurrent Neural Networks (RNNs)**
  - Include feedback loops that pass information from previous steps
  - Maintain memory of past inputs through hidden states
  - Designed for sequential and time-dependent data
  - Effective for tasks where context matters, such as text and time-series analysis

How RNN Differs from Feedforward Neural Networks?



Variants of Recurrent Neural Networks (RNNs)

- RNNs evolved over time because **vanilla RNNs were not powerful enough** to handle long sequences.

  So new improved versions were introduced.
- We have **4 main RNN variations**:
- Vanilla RNN
- Bidirectional RNN
- LSTM
- GRU

1. Vanilla RNN

- This is the *original/basic* RNN.
- **How it works:**
- It has **one hidden layer**.
- It takes the previous hidden state and the current input.
- It does:
- $h_t = \tanh\left(W_x x_t + W_h h_{t-1}\right)$
- **Strength:**
- Learns short-term patterns (like previous few words).
- ✅ **Weakness:**
- Suffers from **vanishing gradient problem**, meaning it **forgets long-term information** (anything far back in the sequence).
- **Good for:**
  - Short sentences
  - Simple sequence patterns
  - Small datasets
- **Example Task:**
  - Predict next character in a short string
  - Small toy sequence tasks

2. Bidirectional RNN (BiRNN)

- A BiRNN reads the sequence **in two directions**:
  - Forward RNN (left → right)
  - Backward RNN (right → left)
- The outputs from both directions are combined.
- **Why this helps:**
- At each time step, the model has:
  - Information from **past**
  - Information from **future**
- This provides a complete context.
- **When to use:**
- When the **entire input sequence is known beforehand**.
- **Example Tasks:**
  - Named Entity Recognition (NER)
  - POS tagging
  - Question answering
  - Speech recognition
  - Handwriting recognition
- ✅ **Not suitable for:**
- ❌ Real-time prediction
- (You don’t know the future tokens yet)

3. LSTM (Long Short-Term Memory)

- **Why LSTMs were created:**
- Vanilla RNNs forget long-term dependencies.
- LSTMs were introduced to **solve the vanishing gradient problem**.
- **How LSTM works:**
- Each LSTM cell contains **three gates**:
  - **Forget Gate** → What to forget
  - **Input Gate** → What new info to add
  - **Output Gate** → What to output
- These gates let the model **control its memory**.
- LSTM = RNN + long-term memory
  LSTMs can remember information from **far back** in a sequence.
- **Strengths:**
  - Excellent for long sequences
  - Can track context over many time steps
  - No vanishing gradient issue (mostly)
- **Where LSTMs are used:**
  - Machine translation
  - Text generation
  - Speech recognition
  - Chatbots
  - Time series forecasting
  - Music generation
- **Example:**
- LSTM can understand a sentence like:
- “The food was tasty, although the service was terrible.”
- It remembers context even after many words.

4. GRU (Gated Recurrent Unit)

- **Why GRUs were created:**
- To simplify LSTMs while keeping similar performance.
- **How GRU works:**
- Combines the **forget gate + input gate** into a single **update gate**
- Has a **reset gate** to control how much past info to keep
- Has fewer parameters than LSTM
- So GRU = LSTM but **simpler and faster**.

- **Strengths:**
  - Faster training
  - Less memory usage
  - Performs close to LSTM
  - Good for medium/long sequences
- **Best for:**
  - Real-time applications
  - When you want the speed of RNN but the power of LSTM
  - When the dataset is not huge

Long Short-Term Memory

Long Short-Term Memory

- Long Short-Term Memory (LSTM) is an improved version of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data.
- It uses a memory cell to store information over time, solving the limitations of traditional RNNs.
- This makes it useful for:
  - **Handles Long Term Dependencies**: Remembers information for longer sequences
  - **Memory Cell**: Stores and updates important information over time
  - **Better than RNN**: Overcomes short term memory limitations
  - **Applications**: Used in language translation, speech recognition and time series forecasting

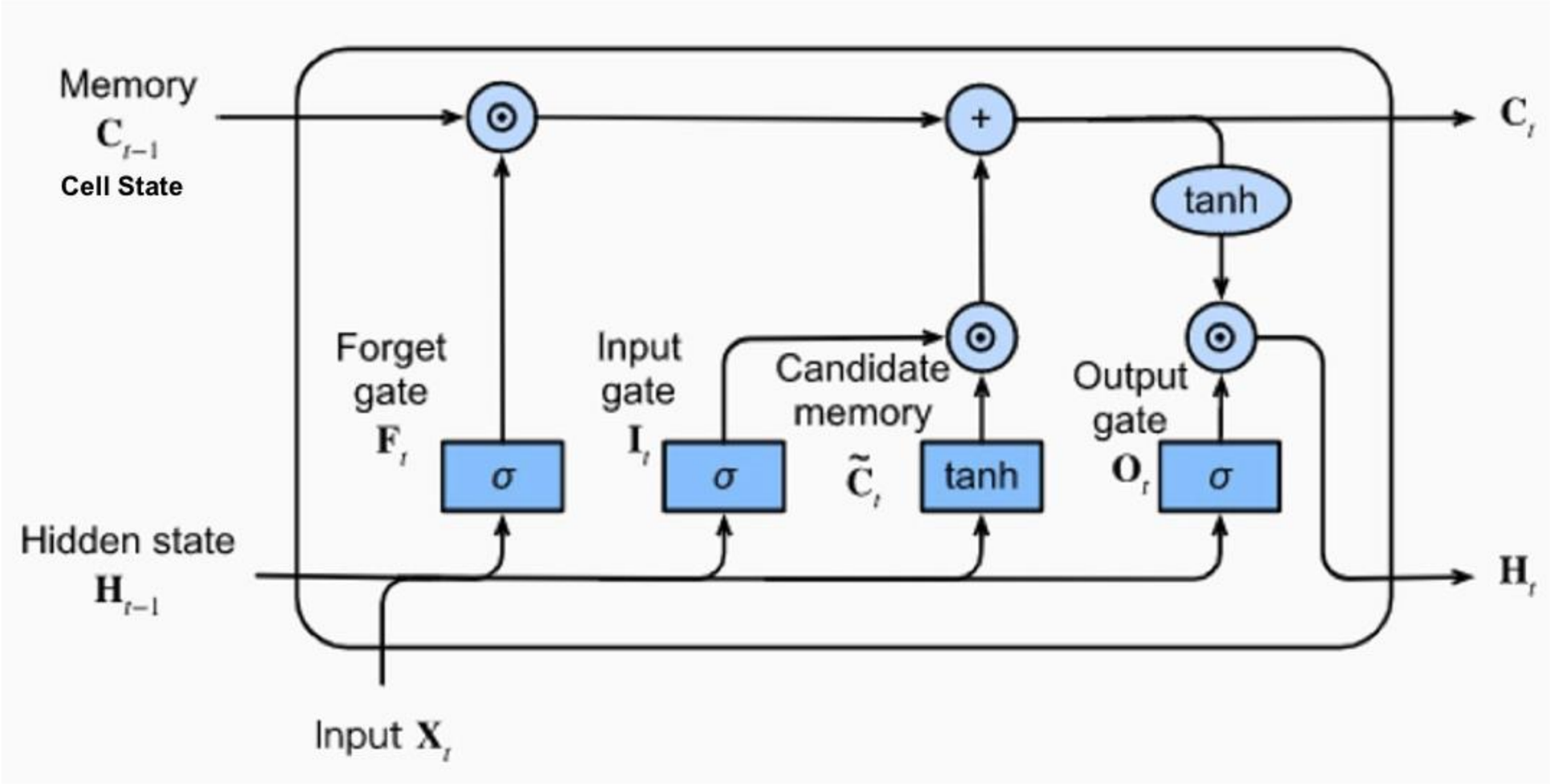
Problem with Long-Term Dependencies in RNN

- RNNs are designed to handle sequential data by using a hidden state that stores information from previous steps.
- However, they struggle to learn long-term dependencies. This happens due to:
- **Vanishing Gradient**:
- When training a model over time, the gradients which help the model learn can shrink as they pass through many steps.
- This makes it hard for the model to learn long-term patterns since earlier information becomes almost irrelevant.
- **Exploding Gradient**:
- Sometimes gradients can grow too large causing instability.
- This makes it difficult for the model to learn properly as the updates to the model become erratic and unpredictable.

LSTM Gates & States

- **1. Forget Gate f_t**
  - Decides what part of the previous memory (cell state) should be forgotten.
- **2. Input Gate i_t**
  - Decides how much new information should be added to the memory.
- **3. Candidate Cell State $\tilde{c}_t$**
  - The new information proposed to be added to the memory.
- **4. Output Gate o_t**
  - Decides how much of the current memory should become the visible output (hidden state).
- **5. Cell State c_t**
  - The LSTM's long-term memory that carries important information across many time steps.
- **6. Hidden State h_t**
  - The LSTM's short-term output that is passed to the next step or next layer.

LSTM Architecture



LSTM Example

- For **each word** (x_1 = "I", x_2 = "love", x_3 = "learning"), the LSTM processes gates in the **same order**.
  - **Forget Gate**
  - **Input Gate**
  - **Candidate Cell**
  - **Update Cell State**
  - **Output Gate**
  - **Update Hidden State**
- **1. Forget Gate (FIRST)**
  - **"What should I forget from the old memory c_{t-1}?"**
  - $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$
    FIRST, the LSTM decides how much of the old memory to keep.
  - Example: "When reading **love**, how much should I keep from memory of **I**?"
- **2. Input Gate (SECOND)**
  - **"Should I allow new information to enter my memory?"**
  - $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$
  - SECOND, it decides if new information should be stored.
- **3. Candidate Cell (THIRD)**
  - **"What is the NEW meaning from the current word?"**
  - $\tilde{c}_t = \tanh W_c x_t + U_c h_{t-1} + b_c)$
  - THIRD, LSTM creates the new knowledge extracted from **current input**.
  - Example: From "learning", extract its meaning.
- **4. Update Cell State (FOURTH)**
  - **"New memory = (Forget old) + (Add new)"**
  - $c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$
  - FOURTH, LSTM updates long-term memory.
  - This is the most important step.
- **5. Output Gate (FIFTH)**
  - **"How much of my memory should I reveal to the outside?"**
  - $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$
    FIFTH, LSTM decides what part of memory becomes visible.
- **6. Hidden State (LAST)**
  - **"This is the output for this time step."**
  - $h_t = o_t \cdot \tanh(c_t)$
  - LAST, LSTM produces the output that goes to:
  - next LSTM cell ($h_t \rightarrow h_{t+1}$), or classifier (sentiment, translation, etc.)

LSTM Example

- In an LSTM cell, there are **two types of weight matrices**:

1. Weights for the input x_t

- These are written as:
- W_f: forget gate weights for input
- W_i: input gate weights for input
- W_c: candidate cell weights for input
- W_o: output gate weights for input
- These connect **current input → gate**
- **Example:** $W_f x_t$

2. Weights for the previous hidden state h_{t-1}

- These are written as:
- U_f: forget gate weights for hidden state
- U_i: input gate weights for hidden state
- U_c: candidate cell weights for hidden state
- U_o: output gate weights for hidden state
- These connect **previous hidden output → gate**

Example: $U_f h_{t-1}$

3. Bias terms

- b_f, b_i, b_c, b_o: These shift the activation.

LSTM Formulas

- **1. Forget Gate:**
  - "What to Forget from Previous Cell State"
- **Purpose:** decide *how much* of the **past memory** c_{t-1} to keep.
- Output range: 0 to 1.

$$\underbrace{f_t}_{\text{Forget Gate Output}} = \sigma\Big(\underbrace{W_f x_t}_{\text{Current Input Contribution}} + \underbrace{U_f h_{t-1}}_{\text{Previous Hidden State Contribution}} + \underbrace{b_f}_{\text{Bias}} \Big)$$

LSTM Formulas

- **2. Input Gate:**
  - “What New Information to Add”
- **Purpose:** controls *how much new information* enters the cell.

$$\underbrace{i_t}_{\text{Input Gate Output}} = \sigma\Big(\underbrace{W_i x_t}_{\text{From Current Input}} + \underbrace{U_i h_{t-1}}_{\text{From Previous Hidden State}} + \underbrace{b_i}_{\text{Bias}} \Big)$$

LSTM Formulas

3. Candidate Cell State

- New Candidate Memory

- **Purpose:** create **new content** that may be added to the cell state.
- **Range:** −1 to 1.

$$\underbrace{\tilde{c}_t}_{\text{Candidate Cell State}} = \tanh\Big(\underbrace{W_c x_t}_{\text{From Current Input}} + \underbrace{U_c h_{t-1}}_{\text{From Previous Hidden State}} + \underbrace{b_c}_{\text{Bias}} \Big)$$

LSTM Formulas

- **4. Output Gate**
  - “What Part of Cell State to Output”
- Purpose: controls what part of the cell state becomes **visible** as hidden state.

$$\underbrace{o_t}_{\text{Output Gate Output}} = \sigma\Big(\underbrace{W_o x_t}_{\text{Current Input Influence}} + \underbrace{U_o h_{t-1}}_{\text{Previous Hidden Influence}} + \underbrace{b_o}_{\text{Bias}} \Big)$$

LSTM Formulas

- **5. Cell State Update:**
  - “New Long-Term Memory”
- **Purpose:** maintain & update the **long-term memory**.

$$\underbrace{c_t}_{\text{Updated Cell State}} = \underbrace{f_t \cdot c_{t-1}}_{\text{Keep Part of Old Memory}} + \underbrace{i_t \cdot \tilde{c}_t}_{\text{Add New Candidate Memory}}$$

LSTM Formulas

- **6. Hidden State Update**
  - “Short-Term Output / Visible State”
- **Purpose:** produce the **output** for time step t.
- This is what is passed to the next LSTM cell and possibly to the next layer.

$$\underbrace{h_t}_{\text{Updated Hidden State}} = \underbrace{o_t}_{\text{Output Gate}} \cdot \underbrace{\tanh(c_t)}_{\text{Activated Cell State}}$$

LSTM Formulas Summary

Gate / State	Uses Input $\mathbf{x_t}$	Uses Hidden $\mathbf{h_{t-1}}$	Uses Cell $\mathbf{c_{t-1}}$	Output
Forget Gate f_t	✅	✅	❌	How much to forget
Input Gate i_t	✅	✅	❌	How much new info to add
Candidate Cell $\tilde{c}_t$	✅	✅	❌	Potential new memory
Output Gate o_t	✅	✅	❌	How much to reveal
Cell State c_t	❌	❌	✅	New long-term memory
Hidden State h_t	❌	✅ indirectly	✅	Output for this step

LSTM Formulas Summary

- An LSTM has **4 gates**:
- **Forget Gate**
  - $f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right)$
- **Input Gate**
  - $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$
- **Candidate Cell State**
  - $\tilde{c}_t = \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right)$
- **Output Gate**
  - $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$
- **Update Cell State**
  - $c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$
- **Update Hidden State**
  - $h_t = o_t \cdot \tanh\left(c_t\right)$

LSTM Example

- We will use a **tiny LSTM**, with:
- Input size = 1
- Hidden size = 1
- Sequence length = 3
- Inputs:
  - x1 = 1.0
  - x2 = 2.0
  - x3 = 3.0
- Initial hidden & cell states:
  - h0 = 0
  - c0 = 0
- To keep math simple, let's choose small weights.
- **Choose Easy Weights**

- Wf = 0.5 Uf = 0.1 bf = 0
- Wi = 0.6 Ui = 0.1 bi = 0
- Wc = 0.7 Uc = 0.1 bc = 0
- Wo = 0.8 Uo = 0.1 bo = 0
- **We will compute:**
  - ft (forget gate)
  - it (input gate)
  - $\hat{c}t$ (candidate cell)
  - ct (updated memory)
  - ot (output gate)
  - ht (hidden output)

LSTM Example: t=1

- **Time Step t = 1 (x_1 = 1.0)**
- **Forget Gate**
  - $\boldsymbol{f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)}$
  - $f_1 = \sigma(0.5(1) + 0.1(0)) = \sigma(0.5)$
  - $f_1 = 0.622$
- **Input Gate**
  - $\boldsymbol{i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)}$
  - $i_1 = \sigma(0.6(1) + 0.1(0)) = \sigma(0.6)$
  - $i_1 = 0.645$
- **Candidate Cell**
  - $\boldsymbol{\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)}$
  - $\tilde{c}_1 = \tanh(0.7(1) + 0.1(0)) = \tanh(0.7)$
  - $\tilde{c}_1 = 0.604$
- **Update Cell State**
  - $\boldsymbol{c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t}$
  - $c_1 = f_1 \cdot c_0 + i_1 \cdot \tilde{c}_1$
  - $c_1 = 0 + (0.645)(0.604) = 0.389$
- **Output Gate**
  - $\boldsymbol{o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)}$
  - $o_1 = \sigma(0.8(1) + 0.1(0)) = \sigma(0.8)$
  - $o_1 = 0.690$
- **Hidden State**
  - $\boldsymbol{h_1 = o_1 \cdot \tanh(c_1)}$
  - $h_1 = 0.690 \cdot \tanh(0.389)$
  - $h_1 = 0.690 \cdot 0.370 = 0.255$
- **Output at t=1:**
  - h1 = 0.255
  - c1 = 0.389

LSTM Example: t=2

- **Time Step t = 2 (x_2 = 2.0)**
- **Forget Gate**
  - $f_2 = \sigma(0.5(2) + 0.1(0.255))$
  - $f_2 = \sigma(1.025) = 0.736$
- **Input Gate**
  - $i_2 = \sigma(0.6(2) + 0.1(0.255)) = \sigma(1.225)$
  - $i_2 = 0.772$
- **Candidate Cell**
  - $\tilde{c}_2 = \tanh(0.7(2) + 0.1(0.255)) = \tanh(1.425)$
  - $\tilde{c}_2 = 0.890$
- **Update Cell**
  - $c_2 = (0.736)(0.389) + (0.772)(0.890)$
  - $c_2 = 0.286 + 0.686 = 0.972$
- **Output Gate**
  - $o_2 = \sigma(0.8(2) + 0.1(0.255)) = \sigma(1.625)$
  - $o_2 = 0.835$
- **Hidden State**
  - $h_2 = 0.835 \cdot \tanh(0.972)$
  - $h_2 = 0.835 \cdot 0.749 = 0.625$
- **Output at t=2:**
  - h2 = 0.625
  - c2 = 0.972

LSTM Example: t=3

- **Time Step t = 3 (x_3 = 3.0)**
- **Forget Gate**
  - $f_3 = \sigma(0.5(3) + 0.1(0.625))$
  - $f_3 = \sigma(1.562) = 0.827$
- **Input Gate**
  - $i_3 = \sigma(0.6(3) + 0.1(0.625))$
  - $i_3 = \sigma(1.862) = 0.865$
- **Candidate Cell**
  - $\tilde{c}_3 = \tanh(0.7(3) + 0.1(0.625)) = \tanh(2.125)$
  - $\tilde{c}_3 = 0.972$
- **Update Cell State**
  - $c_3 = (0.827)(0.972) + (0.865)(0.972)$
  - $c_3 = 0.804 + 0.840 = 1.644$
- **Output Gate**
  - $o_3 = \sigma(0.8(3) + 0.1(0.625)) = \sigma(2.425)$
  - $o_3 = 0.918$
- **Hidden State**
  - $h_3 = 0.918 \cdot \tanh(1.644)$
  - $h_3 = 0.918 \cdot 0.928 = 0.852$
- **Output at t=3:**
  - h3 = 0.852
  - c3 = 1.644

Final Results Summary

Time Step	Input	c_t (Cell State)	h_t (Hidden Output)
t = 1	1.0	0.389	0.255
t = 2	2.0	0.972	0.625
t = 3	3.0	1.644	0.852